DevOps Explanation Request

Alright vicky 😈
This is **DAY-22 energy** — the topic that *decides who gets in* and *who gets fired*.

I'll explain **Authentication vs Authorization** using:

- fortress analogy (because it actually sticks),
- real `kubectl` commands,
- raw API calls,
- and **exactly what happens inside Kubernetes**.

No fluff. No hand-waving.

☠️ Way (If Kubernetes had NO AuthN / AuthZ)

- Anyone with network access → full cluster access
- Pods deleted accidentally (or intentionally)
- etcd wiped → cluster gone 💀

So Kubernetes says:

> "First tell me **who you are**, then I'll decide **what you can do**."

🛠️ How Kubernetes Solves This

Kubernetes uses **two completely separate steps**:

```sql
Authentication → Authorization
```

Order matters.
If Authentication fails → Authorization is NEVER checked.

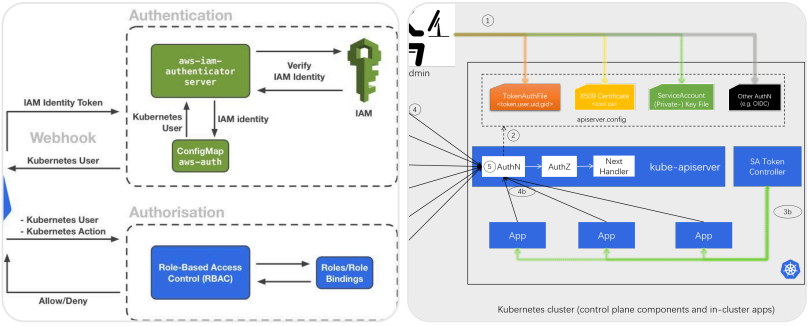
Step 1 – High-level Overview

🔐 Authentication (WHO you are)

> “Are you really vicky, or just pretending?”

✅ Authorization (WHAT you can do)

> “Okay vicky… but are you allowed to delete pods?”



Step 2 – Authentication (WHO YOU ARE)

Think of this as ID check at the gate

Kubernetes API server supports multiple **authentication methods**:

🔑 Authentication Methods

Method	Meaning
Symmetric key	Same secret on both sides
Asymmetric key	Public / Private key
Certificates	Most common (TLS certs)
Tokens	ServiceAccounts

👉 **For humans/admins → CERTIFICATES**
👉 **For pods → TOKENS**

🔐 Kubeconfig = Your Identity Card

Your **kubeconfig** contains:

- API server address
- CA certificate
- Your client certificate
- Your private key

That's why this works:

```bash
kubectl get pods
```

Because kubectl automatically uses:

```text
$HOME/.kube/config
```

🧪 Explicit Authentication Example

Using a specific kubeconfig

```bash
kubectl get pods --kubeconfig config
```

What happens internally:

1. kubectl reads cert + key from `config`
2. Sends request to API server
3. API server verifies cert using CA
4. Authentication SUCCESS or FAIL

🧪 Raw API Call (No Magic)

This command removes **all kubectl abstraction**:

```bash
kubectl get --raw /api/v1/namespaces/default/pods \
  --server https://localhost:64418 \
```

```
  --client-key adam.key \
  --client-certificate adam.crt \
  --certificate-authority ca.crt
```

What EACH FILE means

File	Meaning
`adam.key`	🔐 PRIVATE KEY
`adam.crt`	📜 CLIENT CERTIFICATE
`ca.crt`	📜 CA CERT (trust anchor)

If **any one is wrong** → ❌ Authentication fails

🧠 Authentication Result

If successful, API server now knows:

```ini
username = adam
groups   = system:masters (if present)
```

But… **still NO permission yet** 👇

Step 3 – Authorization (WHAT YOU CAN DO)

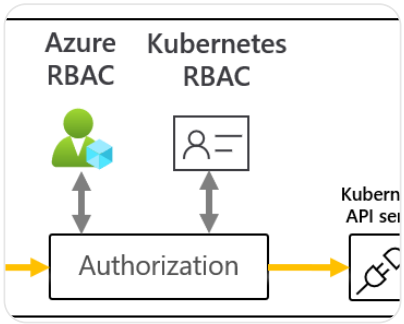
Think of this as access inside the fortress

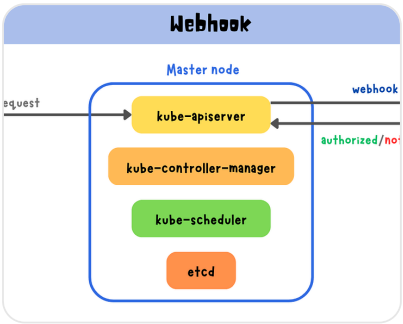
> "You are inside, but where can you go?"

🔑 Authorization Modes in Kubernetes

Mode	Purpose
Node	Node ↔ API server

Mode	Purpose
ABAC	Attribute based (legacy)
RBAC	Role based (RECOMMENDED)
Webhook	External decision engine





🥇 RBAC (REAL WORLD STANDARD)

RBAC answers:

- Can this user **get pods**?
- Can this user **delete deployments**?
- In **which namespace**?

🧠 Authorization Flow (VERY IMPORTANT)

Kubernetes checks authorization **IN ORDER**:

1️⃣ Node Authorizer

→ Is this a kubelet?

→ If yes, allow node operations

2️⃣ RBAC

→ Match user/group with role bindings

3️⃣ Webhook (optional)

→ External policy engines (OPA, Kyverno)

If **any one says ALLOW** → request succeeds ✅

Step 4 – Example: AuthN + AuthZ Together

Scenario

User `adam` runs:

```bash
kubectl delete pod nginx
```

What Kubernetes does

Step 1 – Authentication

- Cert valid? ✅
- Signed by trusted CA? ✅
- Username = `adam`

Step 2 – Authorization

- Is adam allowed to delete pods?
  - RBAC role exists? ×
  - RoleBinding exists? ×

👉 Result:

```javascript
Error: forbidden
```

Authentication PASSED

Authorization FAILED

Step 5 – RBAC Example (Simple)

Role (what actions)

```yaml
```

```yaml
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: pod-reader
  namespace: default
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
```

RoleBinding (who gets it)

```yaml
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: read-pods
  namespace: default
subjects:
- kind: User
  name: adam
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Now:

```bash
kubectl get pods   ✅
kubectl delete pod ❌
```

Step 6 – Common Interview Kill-Shots 💥

× Authentication ≠ Authorization

× RBAC does NOT identify users

× Certificates do NOT give permissions

× system:masters bypasses RBAC (full admin)

× Pods never use certificates (they use tokens)

Step 7 – Final Mental Model (Tattoo-worthy)

```sql
AUTHENTICATION → Who are you?
AUTHORIZATION  → What can you do?
```

Or even simpler:

```sql
Certs / Tokens → Identity
RBAC           → Permission
```

If you want next:

- Full **AuthN + AuthZ request lifecycle**
- Why `system:masters` is dangerous
- ServiceAccount auth deep dive
- Debugging “forbidden” vs “unauthorized”

Say the word 😏🔥